
Principles of Programming Languages:

Problem Sheet 1 – First steps

Mike Spivey, Michaelmas Term 2012.

- 1 Define the functions `foldl` and `foldr` in `Fun`, and use one of them to define `map`:

```
>>> map(lambda (x) x * x, 3:1:4:1:5:nil);;  
--> [9, 1, 16, 1, 25]
```

- 2 The function $index :: Eq\ a \Rightarrow a \rightarrow [a] \rightarrow Int$ is to be defined so that $index\ x\ xs$ is the index (counting from zero) where x first appears in xs , or -1 if x does not appear in xs at all.

- (a) Define this function in Haskell using recursion. Now try to find another definition that does not examine the result of a recursive call to see if it is -1 (or anything similar).
- (b) Translate your definition from part (a) into an (uncurried) `Fun` program.
- (c) Can you find a Haskell definition of $index$ in terms of list comprehensions or standard functions that doesn't directly use recursion?

- 3 Show the abstract syntax trees corresponding to the following `Fun` expressions:

```
3  
x  
x+3  
(x+3)*4  
let val x = 2 in x+3  
let val f(x) = x+3 in f(2)  
f(g(x))  
f(g)(x)
```

- 4 Is the following program legal in `Fun`?

```
let val f(x) =  
  (let val g(y) = x + y in g(2)) in f(3)
```

If so, explain how the reference to x from the body of g is implemented in our interpreter.

2 Principles of Programming Languages: Problem sheet 1

5 Is the following program legal in Fun? If so, what is its value?

```
let val f(x) =  
  let val g(y) = x + y in  
  let val h(x) = g(x + 3) in  
  h(2 * x) in  
f(7)
```

6 Explain the following behaviour of the Fun interpreter.

```
>>> val x = 3;; -- Set x to 3  
--- x = 3  
>>> val f(y) = x + y;; -- f depends on x  
--- f = <function>  
>>> f(2);;  
--> 5  
>>> val x = 4;; -- Change x to 4  
--- x = 4  
>>> f(2);; -- f is the same as before  
--> 5
```

7 In our interpreter for Fun, what happens if a function is called with too many or too few parameters? What would happen if special steps were not taken to produce this behaviour?

What happens if a function definition uses the same name for more than one formal parameter, as in the following program?

```
val f(x, y, x) = x + y
```

Is this behaviour sensible? Is it appropriate if we want to use our interpreter as the definition of Fun?

8 A programming language construct is called *syntactic sugar* if it has the same meaning as another, possibly longer, construct, so that it can be implemented by replacing it syntactically by the longer construct, rather than by building it into the interpreter. By considering the action of both in the Fun interpreter, show that the construct,

```
let val x = e1 in e2,
```

can be regarded as syntactic sugar for

```
(lambda (x) e2)(e1).
```

9 For simplicity, Fun allows only a single definition following `let`. Sometimes it is convenient to allow several definitions at once, and sometimes it is essential, where it is necessary to define recursively several functions that may call each other. Explain how to extend Fun and its interpreter with the following three variants of `let`:

- `let val x1 = e1 and ... and xn = en in e`, where the expressions e_i are all evaluated in the initial environment, and e is evaluated in an environment where the x_i 's have been bound to the values of the corresponding e_i 's.

- let seq $x_1 = e_1$ and $\dots$ and $x_n = e_n$ in e , where each successive e_i is evaluated in an environment where $x_1, \dots, x_{i-1}$ have already been bound. Thus this form is syntactic sugar for

let val $x_1 = e_1$ in let val $x_2 = e_2$ in $\dots$ let val $x_n = e_n$ in e

- let rec $x_1 = e_1$ and $\dots$ and $x_n = e_n$ in e , where the e_i 's must all be lambda expressions, and each of them may contain calls to any of the x_j 's.

10 For compactness of the language, Fun does not have explicit expressions for creating lists, although the interpreter used in Lab 1 does have a primitive list, defined so that $\text{list}(x_1, \dots, x_n)$ returns the list with elements $x_1, \dots, x_n$. Other functional languages provide explicit lists as a construct of the language instead of a primitive function, so that one can write $[x_1, \dots, x_n]$ for the same list. Show how to add this feature to the Fun interpreter.